

Implementacion de Chat Cifrado Extremo a Extremo con ECDH y AES-256-GCM

Autores: Andres Bueno · Sebastian Erazo · Rony Ordoñez

Curso: Seguridad en Redes

Fecha: Junio 2026

Repositorio: [encrypted-chat-java](#)

Tabla de contenidos

1. [Resumen ejecutivo](#)
 2. [Introduccion y objetivos](#)
 3. [Marco teorico](#)
 4. [Arquitectura de la solucion](#)
 5. [Implementacion](#)
 6. [Observador de sesion](#)
 7. [Despliegue con Docker](#)
 8. [Resultados y verificacion](#)
 9. [Conclusiones](#)
 10. [Referencias](#)
-

1. Resumen ejecutivo

Se diseño e implementó una aplicación de chat cifrado peer-to-peer sobre TCP en Java puro, sin ninguna librería criptográfica externa. La aplicación garantiza confidencialidad, integridad, autenticidad y Perfect Forward Secrecy mediante una cadena de primitivas criptográficas: intercambio de claves ECDH sobre la curva secp256r1, derivación de claves con HKDF-SHA-256 (RFC 5869), cifrado autenticado AES-256-GCM y confirmación de claves via HMAC-SHA-256.

Como complemento pedagogico, se construyo un observador de sesion independiente que recibe eventos del protocolo en tiempo real via UDP y los presenta en dos vistas paralelas: la vista criptografica (todos los valores internos) y la vista del atacante (exactamente los bytes que viajan por la red). La aplicacion se contenerizo con Docker para permitir pruebas reproducibles en multiples maquinas sin requerir instalacion de Java en los equipos de destino.

2. Introduccion y objetivos

2.1 Contexto y motivacion

Las comunicaciones de red no cifradas son intrínsecamente inseguras. Un atacante con acceso a la red puede realizar:

- **Sniffing pasivo:** leer todo el trafico en texto plano sin dejar rastro
- **Ataque Man-in-the-Middle (MITM):** interceptar y modificar mensajes en transito, haciendose pasar por cada extremo ante el otro
- **Replay attacks:** capturar mensajes legitimos y reenviarlos posteriormente

El protocolo TCP por si solo no ofrece ninguna garantia criptografica. Protocolos como TLS resuelven este problema en produccion, pero su complejidad interna los convierte en una caja negra desde el punto de vista educativo. Este proyecto implementa un protocolo de seguridad equivalente desde cero, sobre las primitivas del JDK de Java, con el objetivo de hacer visible y explicita cada decision criptografica.

2.2 Objetivo principal

Disenar e implementar un sistema de comunicacion entre dos pares que garantice las siguientes propiedades de seguridad:

Propiedad	Descripcion
Confidencialidad	Ningun tercero puede leer el contenido de los mensajes
Integridad	Cualquier modificacion en transito es detectada y rechazada
Autenticidad	Ambos extremos pueden verificar que hablan con quien creen
Perfect Forward Secrecy	Comprometer claves futuras no permite descifrar sesiones pasadas

2.3 Objetivos especificos

- Implementar el intercambio de claves ECDH sobre la curva estandar secp256r1 usando unicamente `javax.crypto` y `java.security`
- Derivar claves de sesion con distribucion uniforme mediante HKDF-SHA-256 (RFC 5869)
- Cifrar y autenticar todos los mensajes con AES-256-GCM (128-bit tag)
- Verificar que ambos extremos derivaron la misma clave mediante HMAC-SHA-256 antes de cualquier mensaje de chat
- Construir una herramienta de observacion en tiempo real que demuestre la diferencia entre lo que ve un participante legitimo y lo que ve un atacante de red
- Contenerizar la aplicacion con Docker para despliegue reproducible en multiples maquinas

3. Marco teorico

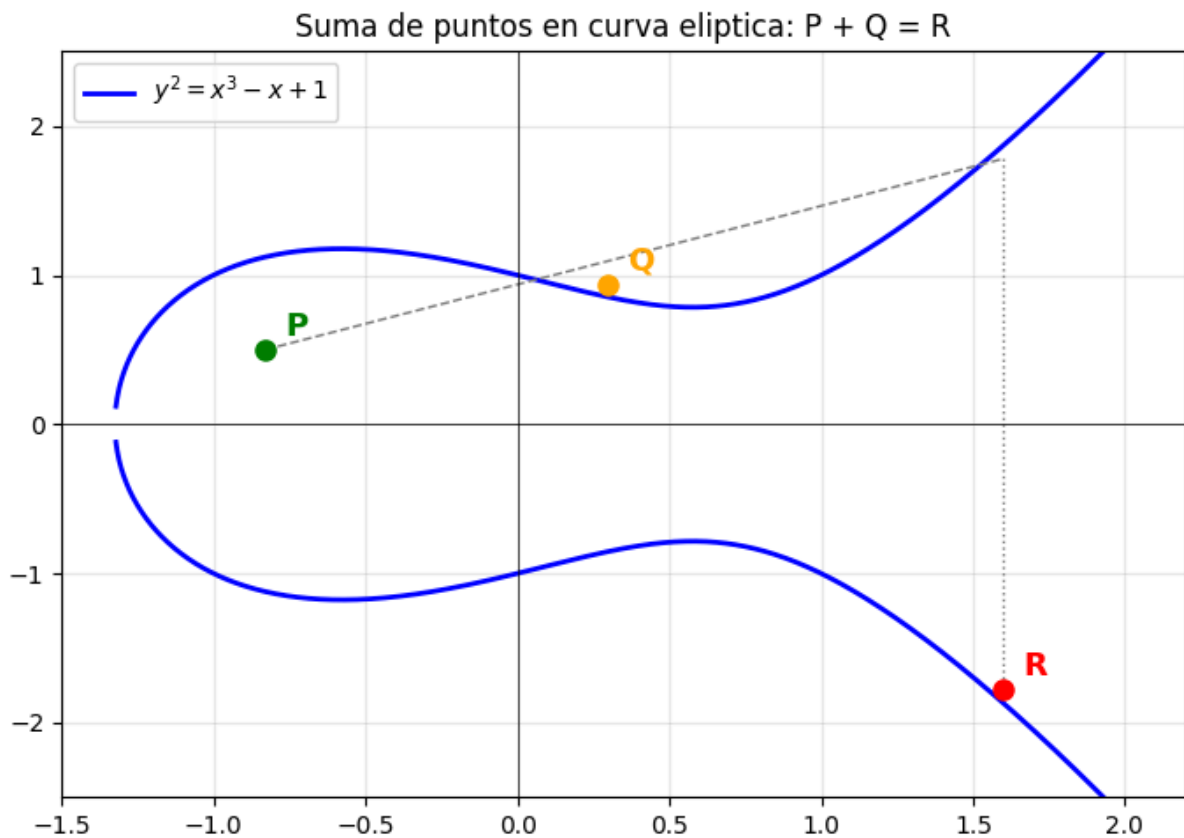
3.1 Criptografia de curva eliptica (ECC)

Una curva eliptica sobre un campo finito se define como el conjunto de puntos que satisfacen la ecuacion:

$$y^2 \equiv x^3 + ax + b \pmod{p}$$

junto con un punto especial en el infinito que actua como elemento neutro. La operacion fundamental es la **multiplicacion escalar**: dado un punto base G y un entero privado k , se calcula $K = kG$ sumando G consigo mismo k veces mediante el algoritmo de duplicacion y adicion.

La seguridad de ECC se basa en el **Problema del Logaritmo Discreto en Curvas Elipticas (ECDLP)**: dado K y G , encontrar k es computacionalmente inviable con los algoritmos conocidos. Para la curva P-256, el mejor ataque conocido requiere del orden de 2^{128} operaciones.



La curva **secp256r1** (también llamada NIST P-256 o prime256v1) tiene los siguientes parametros relevantes:

- Campo primo p de 256 bits
- Orden n del grupo ciclico: aproximadamente 2^{256}
- Punto base G publicamente conocido y estandarizado por el NIST

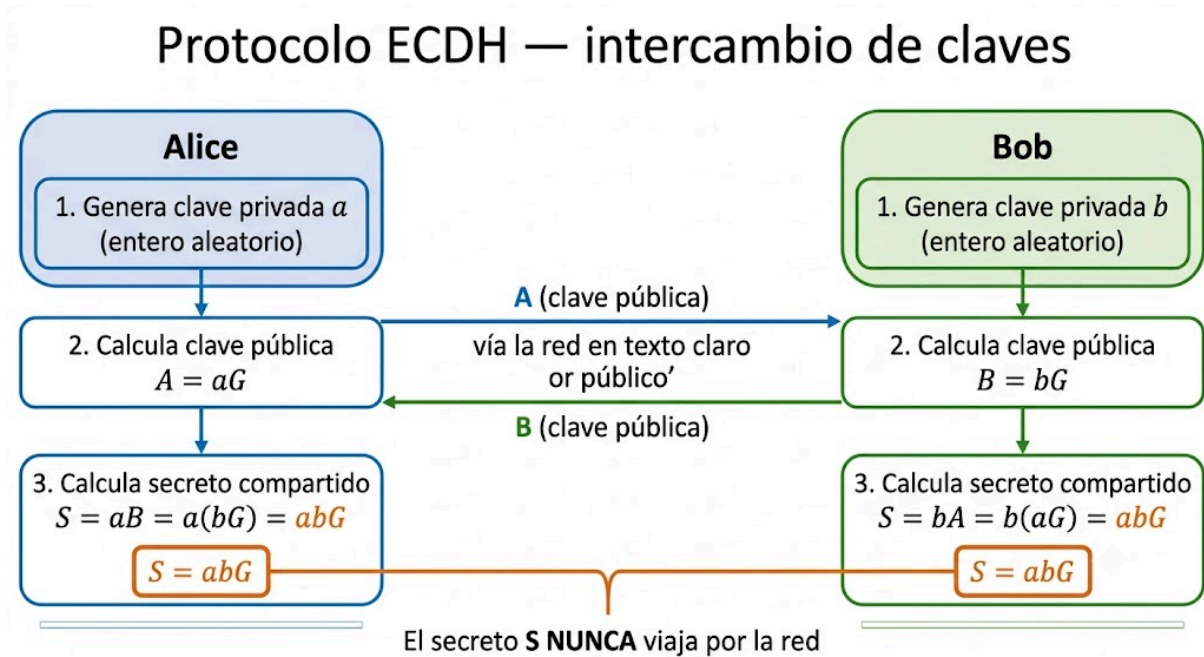
Comparado con RSA, para un nivel de seguridad equivalente a RSA-3072 bits, ECC necesita solo claves de 256 bits, resultando en handshakes mas rapidos, menor uso de memoria y menor overhead de red.

3.2 Intercambio de claves ECDH

El protocolo **Elliptic Curve Diffie-Hellman (ECDH)** permite a dos partes derivar un secreto compartido sin que ningun observador de la red pueda calcularlo, incluso conociendo las claves publicas intercambiadas:

1. Alice genera un par de claves efimero: privada a , publica $A = aG$
2. Bob genera un par de claves efimero: privada b , publica $B = bG$
3. Intercambian sus claves publicas A y B por la red (esto es seguro; solo un observador con ECDLP resuelto podria explotarlo)

4. Alice calcula $S = aB = a(bG) = abG$
5. Bob calcula $S = bA = b(aG) = abG$
6. Ambos obtienen el mismo punto S sin que nadie en la red pueda calcularlo



La palabra **efimero** es clave: en este proyecto se genera un nuevo par de claves EC en cada conexión TCP. Esto da **Perfect Forward Secrecy (PFS)**: incluso si un atacante almacena todo el tráfico cifrado hoy y en el futuro obtiene acceso físico al servidor, no podrá descifrar sesiones pasadas porque las claves privadas que las protegían ya no existen en ningún disco ni memoria.

3.3 Derivación de claves con HKDF-SHA-256

El punto resultante del ECDH no se usa directamente como clave AES por dos razones:

1. Los puntos de una curva elíptica no tienen distribución uniforme como bits aleatorios; tienen estructura matemática que podría ser explotable
2. Se necesitan 64 bytes de material de clave (32 para AES + 32 para confirmación) a partir de 32 bytes de secreto

HKDF (HMAC-based Key Derivation Function, RFC 5869) resuelve ambos problemas en dos fases:

Fase 1 — Extract: transforma el secreto ECDH en una clave pseudoaleatoria uniforme

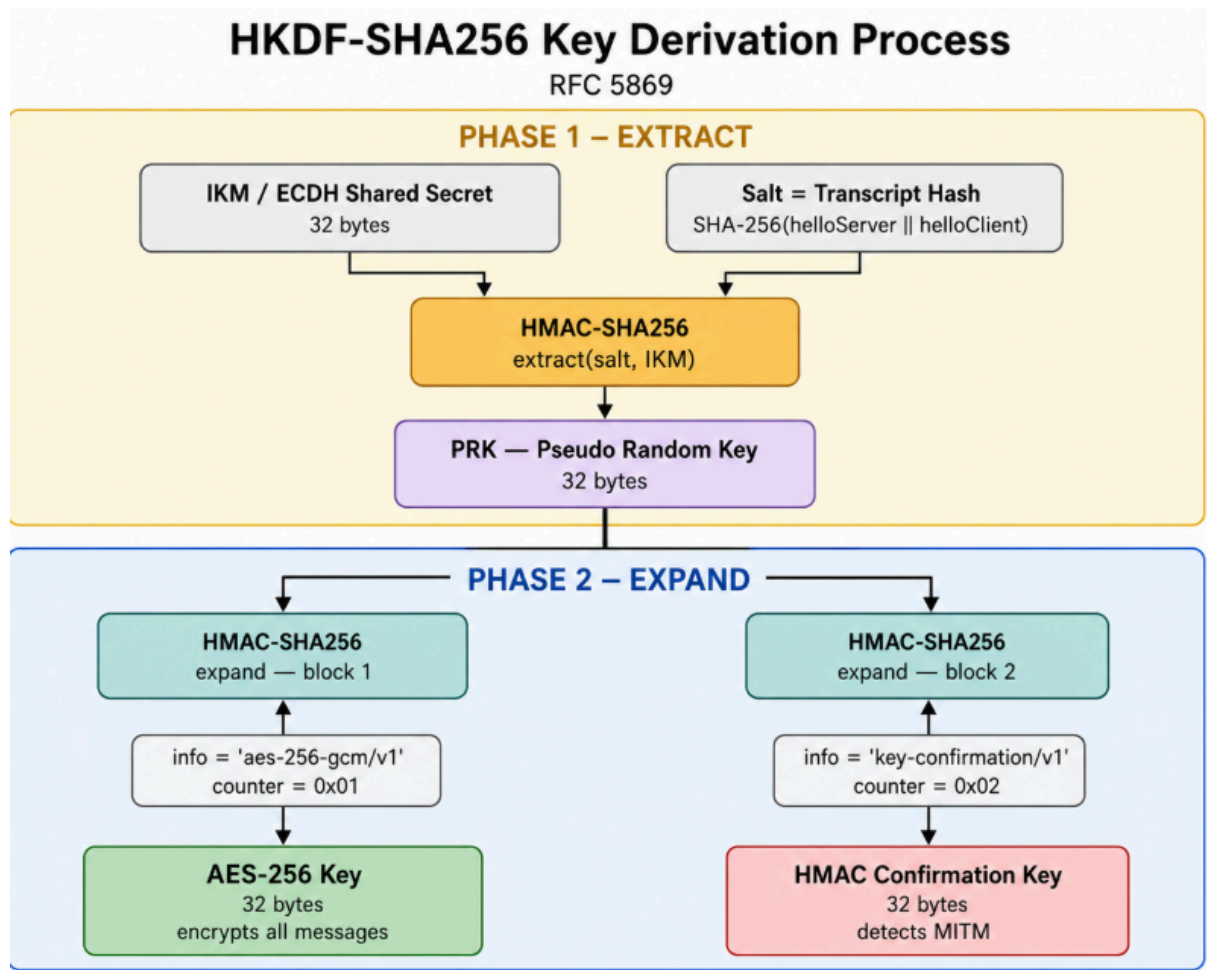
```
PRK = HMAC-SHA256(sal, secreto_ECDH)
```

La **sal** usada es el **transcript hash**: `SHA-256(helloServidor || helloCliente)`. Este valor es unico para cada sesion ya que incluye las claves publicas efimeras. Esto vincula las claves derivadas a esta sesion especifica y previene ataques de replay: si un atacante graba el trafico y lo repite, el transcript hash diferira y las claves derivadas tambien.

Fase 2 — Expand: genera tantos bytes como se necesiten con distribucion uniforme

```
T(1) = HMAC-SHA256(PRK, "" || info || 0x01)
T(2) = HMAC-SHA256(PRK, T(1) || info || 0x02)
OKM = T(1) || T(2) || ...
```

Con `length = 64`: los primeros 32 bytes son la clave AES-256, los siguientes 32 bytes son la clave de confirmacion HMAC.



3.4 Cifrado autenticado AES-256-GCM

AES-256-GCM es un esquema de cifrado **AEAD** (Authenticated Encryption with Associated Data) que combina en una sola operacion cifrado de flujo y autenticacion de integridad, siendo eficiente tanto en hardware como en software.

Componentes:

- **AES-256**: cifrado de bloque con clave de 256 bits (14 rondas de sustitucion-permutacion). Proporciona confidencialidad
- **Modo GCM**: usa AES en modo contador (CTR) para generar un keystream que se combina con XOR con el plaintext, y el campo de Galois $GF(2^{128})$ para calcular el tag de autenticacion de 128 bits

Estructura de un mensaje cifrado:

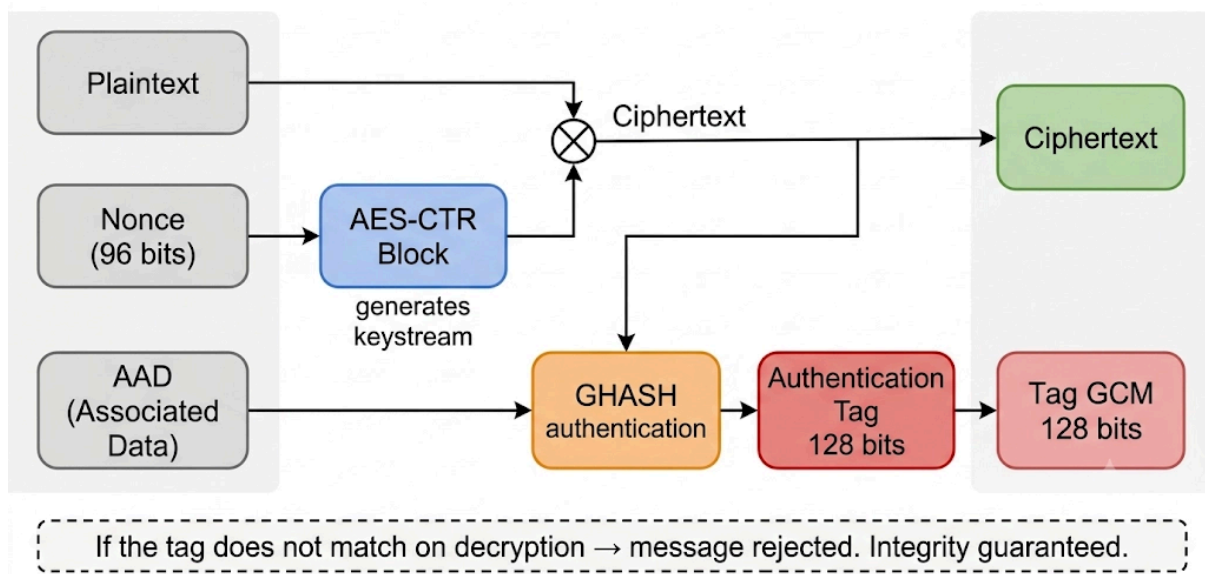
```
Entradas:  plaintext + nonce (96 bits) + clave AES (256 bits) + AAD opcional
Salidas:   ciphertext (mismo tamaño que plaintext) + tag de autenticacion (128 bits)
```

El **nonce** de 96 bits es un requisito critico de seguridad: **reutilizar el mismo nonce con la misma clave rompe completamente la seguridad de GCM**, permitiendo al atacante recuperar el plaintext y la clave de autenticacion. Para evitarlo, se usa un contador monotono de 64 bits.

Separacion de nonces por rol: servidor y cliente usan prefijos distintos en los primeros 4 bytes del nonce:

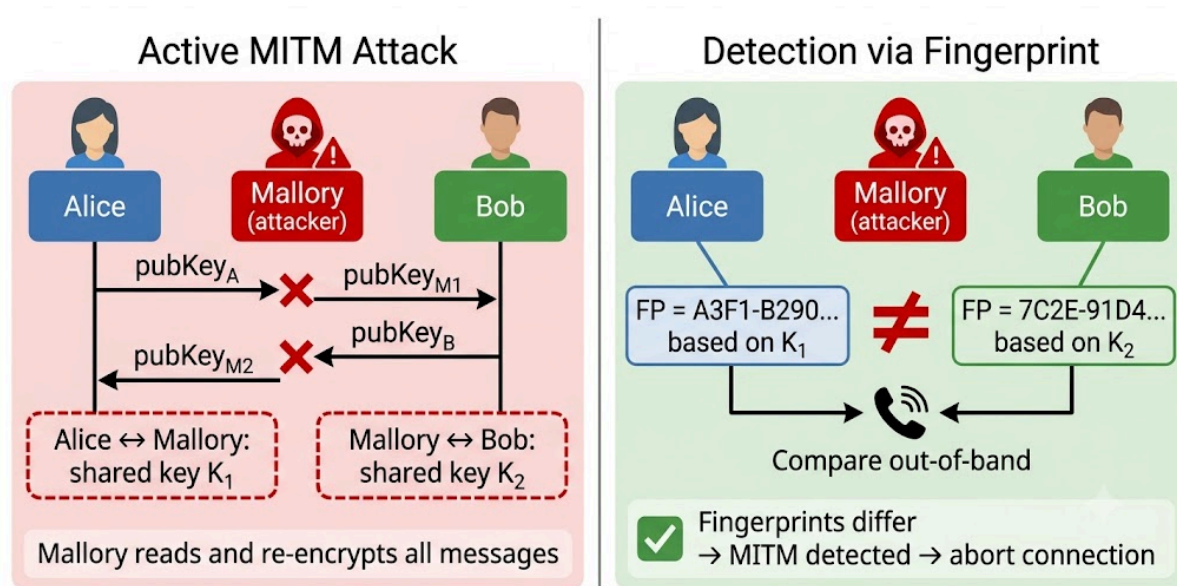
```
Servidor: 0x53324301 | contador 64 bits
Cliente:  0x43325301 | contador 64 bits
```

Esto permite que ambos usen la misma clave AES en ambas direcciones sin ningun riesgo de colision de nonces.



3.5 Confirmacion de claves y fingerprint

ECDH autentica que ambos extremos derivaron el mismo secreto matematico, pero no autentica la **identidad** de los participantes. Un atacante MITM podria interceptar las claves publicas durante el intercambio, sustituirlas por las propias y establecer dos sesiones ECDH separadas, leyendo y re-cifrando todos los mensajes en tiempo real sin que ninguno de los extremos lo detecte.



El protocolo implementa dos mitigaciones complementarias:

a) Confirmacion HMAC: inmediatamente tras derivar las claves, cada extremo calcula:


```
confirmacion = HMAC-SHA256(claveConfirmacion, byte(rol) || transcript)
```

y lo envia al otro. Si ambos derivaron la misma `claveConfirmacion` (lo cual solo ocurre si no hay MITM), los HMACs coinciden. Si hay MITM, son distintos y la conexion se rechaza con error criptografico antes de cualquier mensaje de chat.

b) Fingerprint de seguridad: se calcula como:

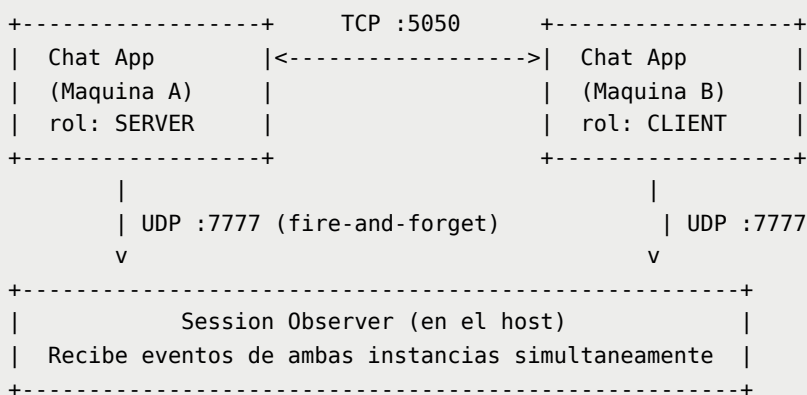
```
hash = SHA-256("encrypted-chat/fingerprint/v1" || transcriptHash || claveAES)
fingerprint = primeros 16 bytes formateados en hex con guiones
Ejemplo: A3F1-B290-4C8E-7D12
```

Este valor se muestra en ambas terminales. Si los usuarios lo comparan por un canal alternativo (telefono, video llamada) y coincide, no hay MITM activo.

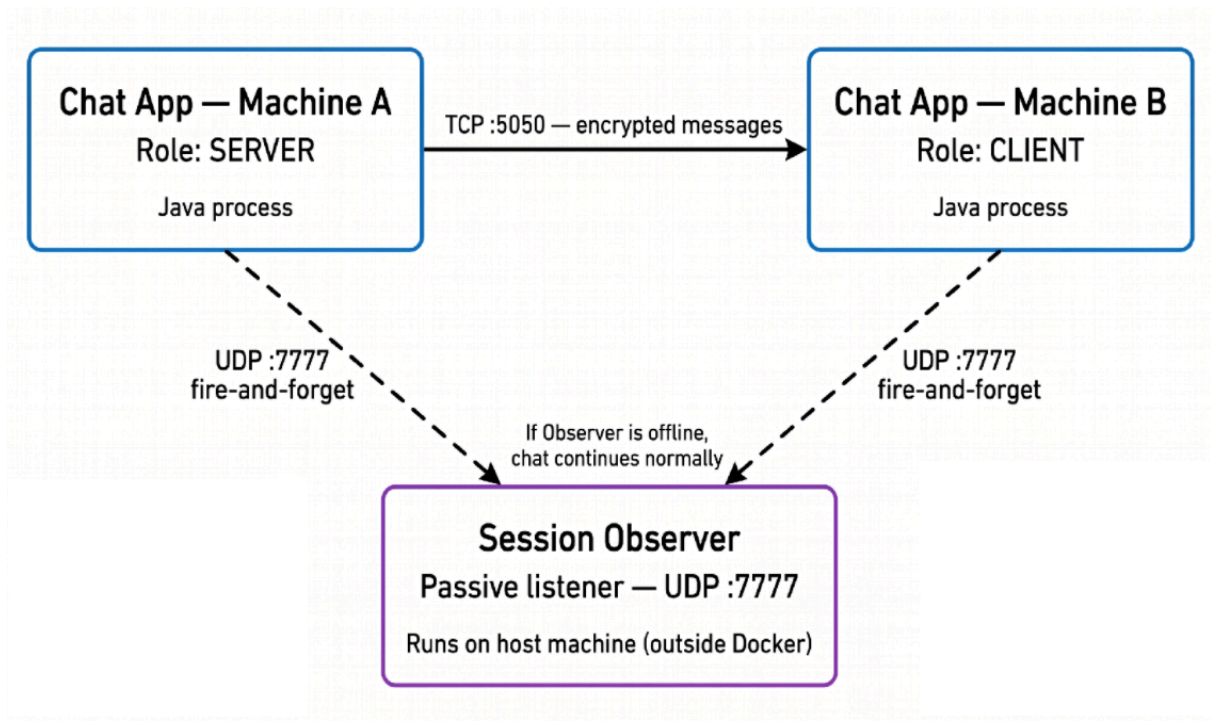
4. Arquitectura de la solucion

4.1 Vision general del sistema

La solucion se compone de tres procesos independientes que se comunican por red:



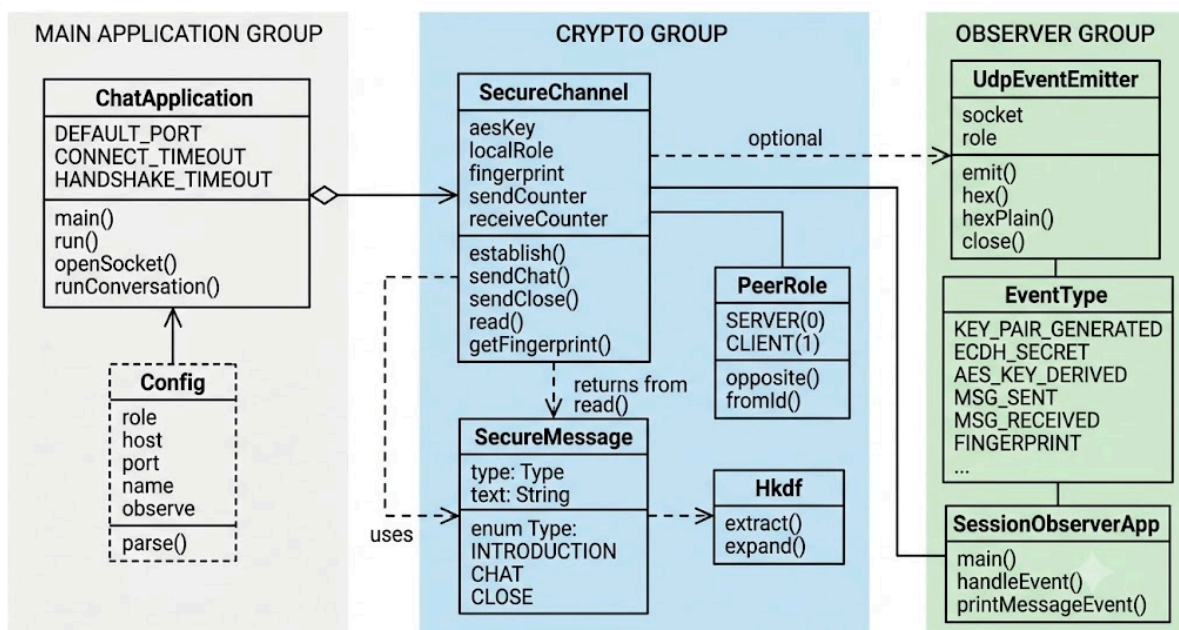
El observer es completamente **pasivo**: si no esta corriendo, las aplicaciones de chat funcionan con normalidad. Si un datagrama UDP se pierde, se ignora silenciosamente. No afecta el rendimiento ni la seguridad del canal cifrado.



4.2 Estructura de clases

Clase	Paquete	Responsabilidad
<code>ChatApplication</code>	<code>com.encryptedchat</code>	Entry point: parseo de argumentos CLI, apertura de socket TCP, loop de conversacion con hilos separados para entrada y salida
<code>SecureChannel</code>	<code>com.encryptedchat.crypto</code>	Ciclo de vida criptografico completo: handshake ECDH, derivacion de claves, confirmacion HMAC, cifrado y descifrado de todos los mensajes. Thread-safe via <code>sendLock</code>
<code>SecureMessage</code>	<code>com.encryptedchat.crypto</code>	Value object inmutable: tipo de mensaje (INTRODUCTION,

Clase	Paquete	Responsabilidad
		CHAT, CLOSE) y payload de texto
PeerRole	com.encryptedchat.crypto	Enum SERVER/CLIENT con IDs de rol numericos usados para separacion de nonces
Hkdf	com.encryptedchat.crypto	Implementacion completa de RFC 5869: extract y expand sobre HMAC-SHA-256
UdpEventEmitter	com.encryptedchat.observer	Serializa eventos como JSON y los envia por UDP al observer. Fire-and-forget: cualquier error se ignora silenciosamente
EventType	com.encryptedchat.observer	Enum de todos los tipos de eventos observables durante handshake y chat
SessionObserverApp	com.encryptedchat.observer	Proceso standalone: escucha UDP :7777, parsea JSON, visualiza eventos con colores ANSI



4.3 Protocolo ECH1 — formato de tramas en la red

El protocolo se identifica con el magic `0x45434831` (bytes ASCII de "ECH1").

Trama HELLO (intercambio de claves publicas durante el handshake):

```

+-----+-----+-----+-----+-----+
| 4 bytes | 1 byte | 1 byte | 4 bytes | N bytes |
| magic   | version | rol    | longitud | clave publica X.509 |
| 0x45434831 | 0x01  | 0/1   |         | (91 bytes tipico)  |
+-----+-----+-----+-----+-----+

```

Trama DATA (todos los mensajes post-handshake):

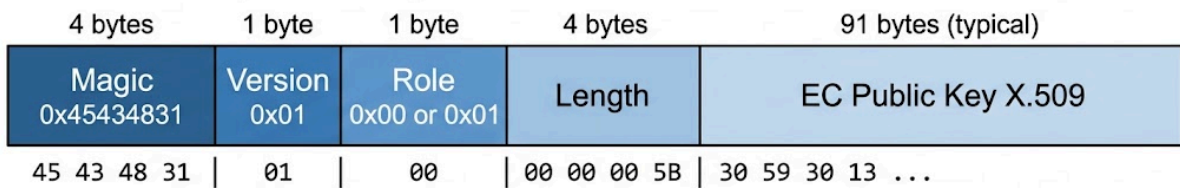
```

+-----+-----+-----+-----+
| 8 bytes | 4 bytes | N bytes | 16 bytes |
| contador (uint64) | longitud | ciphertext | tag GCM |
| 00 00 00 00 00 ... | 00 00 00 17 | 52 58 9C 26 ... | 66 C3 80 ... |
+-----+-----+-----+-----+

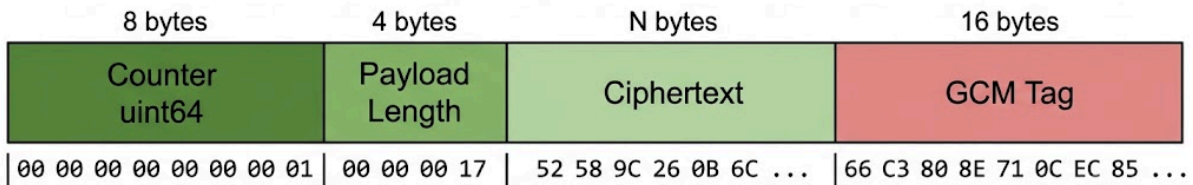
```

El contador va en texto claro porque el receptor lo necesita para reconstruir el nonce. La longitud permite al receptor leer exactamente los bytes correctos antes de intentar descifrar. El ciphertext y el tag son indistinguibles de ruido aleatorio para quien no tiene la clave AES.

HELLO Frame (handshake)



DATA Frame (encrypted messages)



Counter in plaintext (needed for nonce reconstruction).
Ciphertext + Tag are indistinguishable from random bytes without the AES key.

5. Implementacion

5.1 Flujo completo del handshake

```
SERVER                                     CLIENT
|                                         |
|-- genera par de claves EC efimero ---->|-- genera par de claves EC efimero
|                                         |
|----- HELLO (magic+ver+rol+pubKey) --->|
|<----- HELLO (magic+ver+rol+pubKey) ----|
|                                         |
|-- transcript = SHA256(helloS||helloC) --|
|-- secreto = ECDH(privLocal, pubPeer) --|
|-- PRK = HKDF.extract(transcript, secret)|
|-- claveAES(32B) = HKDF.expand(PRK, ...) |
|-- claveHMAC(32B)= HKDF.expand(PRK, ...) |
|                                         |
|----- confirmacion HMAC (cifrada) ---->|
|<----- confirmacion HMAC (cifrada) ----|
|                                         |
|-- verificar HMAC del peer                |-- verificar HMAC del peer
|-- calcular fingerprint                   |-- calcular fingerprint
|                                         |
|===== CANAL SEGURO ESTABLECIDO =====|
|                                         |
|----- INTRODUCTION (nombre) ----->|
|<----- INTRODUCTION (nombre) -----|
|                                         |
|<===== CHAT CIFRADO EN CURSO =====>
```

5.2 Generacion del par de claves EC

```
// SecureChannel.java
private static KeyPair generateKeyPair() throws GeneralSecurityException {
    KeyPairGenerator generator = KeyPairGenerator.getInstance("EC");
    generator.initialize(new ECGenParameterSpec("secp256r1"), new SecureRandom());
    return generator.generateKeyPair();
}
```

Se usa `SecureRandom` para la clave privada, garantizando entropía criptográficamente segura en cada llamada. El nombre estándar `"secp256r1"` es reconocido por el proveedor SunEC del JDK. Cada llamada a `establish()` genera un par nuevo, sin reutilización.

5.3 Implementacion de HKDF (RFC 5869)

```
// Hkdf.java – fase Extract
static byte[] extract(byte[] salt, byte[] inputKeyMaterial)
    throws GeneralSecurityException {
    byte[] actualSalt = salt == null ? new byte[HASH_LENGTH] : salt;
    Mac mac = Mac.getInstance("HmacSHA256");
    mac.init(new SecretKeySpec(actualSalt, "HmacSHA256"));
    return mac.doFinal(inputKeyMaterial);
}

// Hkdf.java – fase Expand
static byte[] expand(byte[] pseudoRandomKey, byte[] info, int length)
    throws GeneralSecurityException {
    Mac mac = Mac.getInstance("HmacSHA256");
    mac.init(new SecretKeySpec(pseudoRandomKey, "HmacSHA256"));
    byte[] output = new byte[length];
    byte[] previous = new byte[0];
    int offset = 0, block = 1;
```

```

while (offset < length) {
    mac.reset();
    mac.update(previous);
    if (info != null) mac.update(info);
    mac.update((byte) block);
    byte[] current = mac.doFinal();
    int copied = Math.min(current.length, length - offset);
    System.arraycopy(current, 0, output, offset, copied);
    offset += copied;
    Arrays.fill(previous, (byte) 0); // borrar bloque anterior de la heap
    previous = current;
    block++;
}
Arrays.fill(previous, (byte) 0);
return output;
}

```

La implementacion es fiel al pseudocodigo del RFC 5869. El borrado de `previous` en cada iteracion reduce el tiempo que el material de clave permanece en la heap de Java.

5.4 Construcción del nonce y cifrado AES-GCM

```

// SecureChannel.java – construcción del nonce de 96 bits
private static byte[] nonceFor(PeerRole sender, long counter) {
    ByteBuffer nonce = ByteBuffer.allocate(12);
    nonce.putInt(sender == PeerRole.SERVER ? 0x53324301 : 0x43325301);
    nonce.putLong(counter);
    return nonce.array();
}

// Cifrado de un mensaje
byte[] nonce = nonceFor(localRole, sendCounter);
Cipher cipher = Cipher.getInstance("AES/GCM/NoPadding");

```



```

cipher.init(Cipher.ENCRYPT_MODE, aesKey, new GCMParameterSpec(128, nonce));
cipher.updateAAD(aadFor(nonce));           // version + nonce como AAD
byte[] encrypted = cipher.doFinal(plaintext);

```

El **AAD** (Associated Authenticated Data) incluye el número de versión del protocolo y el nonce. El tag de autenticación cubre tanto el ciphertext como el AAD, lo que significa que un atacante no puede reutilizar un ciphertext válido cambiando el contador de la trama.

5.5 Borrado seguro de material clave

```

// SecureChannel.java – bloque finally tras derivar las claves
try {
    confirmKey(output, input, confirmationKey, transcript, localRole, emitter);
    SecretKeySpec aesKey = new SecretKeySpec(aesKeyBytes, "AES");
    return new SecureChannel(socket, input, output, aesKey, localRole, fingerprint, emitter);
} finally {
    Arrays.fill(sharedSecret, (byte) 0);
    Arrays.fill(pseudoRandomKey, (byte) 0);
    Arrays.fill(aesKeyBytes, (byte) 0);
    Arrays.fill(confirmationKey, (byte) 0);
}

```

El secreto ECDH, la PRK de HKDF y la clave de confirmación se borran de la heap inmediatamente tras usarlos, independientemente de si ocurre una excepción. Solo persiste el objeto `SecretKeySpec` que encapsula la clave AES de forma opaca en el JDK.

5.6 Confirmación de claves via HMAC

```

// SecureChannel.java – calculo del HMAC de confirmacion
private static byte[] confirmationMac(byte[] key, byte[] transcript, PeerRole role)

```

```

        throws GeneralSecurityException {
            Mac mac = Mac.getInstance("HmacSHA256");
            mac.init(new SecretKeySpec(key, "HmacSHA256"));
            mac.update((byte) role.id()); // diferencia el HMAC de
            cada extremo
            return mac.doFinal(transcript);
        }

```

Incluir el `role.id()` en el HMAC evita que un atacante reenvíe el HMAC recibido del otro lado (ambos calculos son distintos aunque usen la misma clave). Incluir el `transcript` vincula el HMAC a esta sesion especifica.

6. Observador de sesion

6.1 Problema que resuelve

El cifrado de extremo a extremo, por definicion, oculta la informacion. Esto dificulta verificar pedagogicamente que el sistema funciona correctamente. El observador resuelve este problema de forma elegante: ejecutandose en el mismo host que la aplicacion, recibe todos los valores internos del protocolo antes de que sean cifrados o descartados, y los muestra junto con los bytes reales que viajan por la red.

6.2 Arquitectura del observador

```

Chat App (con --observe activo)
|
| 1. Evento criptografico ocurre (ej: AES_KEY_DERIVED)
| 2. UdpEventEmitter serializa como JSON minimo
|   {"role":"SERVER","type":"AES_KEY_DERIVED","key":"3A:F2:..."}
| 3. DatagramSocket.send() a OBSERVER_HOST:7777
|   (fire-and-forget: no espera confirmacion)
|
Session Observer (SessionObserverApp)
|
| 4. DatagramSocket.receive() en :7777
| 5. parseJson() -> Map<String, String>
| 6. handleEvent() selecciona visualizacion segun tipo
| 7. Imprime en terminal con colores ANSI

```

UDP fue elegido deliberadamente: es fire-and-forget, no bloquea al chat si el observer no esta disponible, no afecta el timing criptografico y no requiere que

el observer este activo antes que la aplicacion.

6.3 Tipos de eventos emitidos

EventType	Cuando se emite	Informacion visible
<code>KEY_PAIR_GENERATED</code>	Generacion del par EC	Clave publica en hex (65 bytes)
<code>HELLO_SENT</code> / <code>HELLO_RECEIVED</code>	Intercambio HELLO	Bytes completos del frame
<code>TRANSCRIPT_HASH</code>	Hash de sesion calculado	SHA-256 en hex
<code>ECDH_SECRET</code>	Tras <code>agreement.generateSecret()</code>	Secreto compartido — NUNCA sale por la red
<code>HKDF_EXTRACT</code>	Tras <code>extract()</code>	Pseudo Random Key (PRK)
<code>AES_KEY_DERIVED</code>	Tras <code>expand()</code> primeros 32B	Clave AES de 256 bits
<code>CONFIRMATION_KEY_DERIVED</code>	Tras <code>expand()</code> segundos 32B	Clave de confirmacion HMAC
<code>CONFIRMATION_SENT</code> / <code>RECEIVED</code>	Intercambio de HMACs	HMAC + resultado de verificacion
<code>FINGERPRINT</code>	Fingerprint calculado	Valor formateado
<code>MSG_SENT</code> / <code>MSG_RECEIVED</code>	Por cada mensaje	Texto, nonce, plaintext, ciphertext, tag GCM, frame TCP completo

6.4 Las dos vistas por mensaje

Por cada mensaje de chat, el observer presenta dos cajas claramente separadas:

```
#1 [SERVER] ENVIADO [CHAT]

+--- VISTA CRIPTOGRAFICA (solo visible en este proceso) -----+
| Texto      : "Hola Bruno"
| Nonce      : 53:32:43:01 | 00:00:00:00:00:00:00:01
| Plaintext: 02 48 6F 6C 61 20 42 72 75 6E 6F
| Cifrado   : 7A 3F C1 08 2B 9E 44 D2 F1 C8 A0
| Tag GCM   : 91 B4 2C 7E 03 F5 88 A1 6D 4E B7 22 91 C3 5A 0F
+-----+

+--- VISTA DEL ATACANTE (bytes que viajan por la red) -----+
| Frame TCP (43 bytes):
| contador : 00 00 00 00 00 00 00 01
```

```
| longitud : 00 00 00 1B
| payload  : 7A 3F C1 08 2B 9E 44 D2 F1 C8 A0
| tag GCM   : 91 B4 2C 7E 03 F5 88 A1 6D 4E B7 22 91 C3 5A 0F
+-----+
```

La **Vista criptografica** muestra lo que solo puede ver quien tiene acceso al proceso: el texto en claro, el nonce con su estructura (prefijo de rol + contador), el plaintext en bytes, el ciphertext y el tag. Ninguno de estos valores, excepto el frame completo, existe en la red.

La **Vista del atacante** muestra exactamente los bytes del frame TCP. El atacante con Wireshark puede ver la cantidad de bytes, el contador en claro (necesario para sincronizacion) y los datos cifrados. Sin la clave AES, no puede distinguir el ciphertext del tag, ni recuperar ningun fragmento del texto original.

```
00:50:31 docker run -it -p 5050:5050 -e OBSERVER_HOST=host.docker.internal andresb420
Esperando una conexi3n en 0.0.0.0:5050...
Negociando clave mediante ECDH (secp256r1)...
[--observe activo] enviando eventos al observador en UDP :7777
Canal AES-256-GCM establecido con 172.17.0.1:55692
Codigo de seguridad: C9F1-B080-AC5E-5949-BAA9-350D-2AA1-F44E
Compare este codigo con el otro usuario por un canal distinto.
Conectado con Bruno. Escriba /salir para terminar.
```

```
+-----+
| OBSERVADOR DE SESION COMBINADO |
| Escuchando eventos en UDP :7777 |
+-----+

HANDSHAKE

[SERVER] Clave publica EC generada (A = axG)
30:59:30:13:06:07:2A:86:48:CE:3D:02:01:06:08:2A:86:48:CE:3D:03:01:07:03:42:00:04:21:95:49:BC:54:6D:A0:0D:F6:B6:45:7F:4D:E6:6D:4B:CB:3E:57:D7:32:44:AA:29:7C:27:BD:72:06:C2:E5:6C:E3:47:C0:00:38:FA:DC:97:24:1F:4E:EF:94:54:12:BE:ED:E9:73:77:BD:6B:96:41:5A:BB:1E:C7:1B:1D:CD:76
→ 65 bytes - se envia al otro extremo
[SERVER] Hello enviado
45:43:48:31:01:00:00:00:5B:30:59:30:13:06:07:2A:86:48:CE: ...
→ magic + version + rol + clave publica
| HELLO_SENT [SERVER]: 45:43:48:31:01:00:00:00:5B:30:59:30:1...
[SERVER] Hello recibido
45:43:48:31:01:00:00:00:5B:30:59:30:13:06:07:2A:86:48:CE: ...
→ del otro extremo
| HELLO_RECEIVED [SERVER]: 45:43:48:31:01:00:00:00:5B:30:59:30:1...
[SERVER] Transcript hash (sal HKDF)
B3:3C:D2:5D:4C:24:0D:DB:93:6C:7F:06:52:3D:BD:1A:B7:6C:14:FE:3B:56:EA:12:2E:35:34:0C:F3:50:A2:8C
→ SHA-256 de ambos hellos - unico por sesion

+-----+
| [SERVER] SECRETO ECDH (axbxG) |
| B8:BB:39:F4:67:0C:BE:83:59:6C:0B:57:48:0B:04:54:99:F... |
| [!] Este valor NUNCA sale por la red |
+-----+
```

```

[SERVER] HKDF extract → PRK
7E:39:C2:87:7F:68:2E:9F:16:91:78:98:70:66:F6:83:8A:12:36:B7:38:D2:F5:17:6B:FC:7C:09:6B:B0:54:D9
→ secreto ECDH + sal → bytes uniformes
[SERVER] Clave AES-256 derivada
60:9C:70:6B:13:15:D5:0C:DD:C9:31:F9:24:8E:FB:1B:CF:6F:17:92:90:2C:4B:5B:EC:B5:26:1B:ED:7C:4E:9C
→ 32 bytes - cifra todos los mensajes
[SERVER] Clave de confirmacion derivada
F1:6A:1F:18:5C:D8:6D:28:B8:50:45:C2:D3:05:E2:57:65:7D:E1:C1:94:0D:23:E2:D6:B5:6A:50:03:E1:70:3C
→ 32 bytes - verifica que ambos lados coinciden
[SERVER] HMAC de confirmacion enviado
F5:A3:A9:3E:AE:57:F0:CE:47:AC:33:EF:4C:35:3F:39:9F:E7:85:DF: ...
[SERVER] HMAC de confirmacion recibido
74:F2:6A:37:F1:BC:20:B7:AD:6C:DB:A2:1F:31:FF:2C:9E:A6:AF:60: ...
[OK] CLAVES COINCIDEN - sin MITM detectado

+-----+
| Fingerprint [SERVER]: C9F1-B080-AC5E-5949-BAA9-350D-2AA1-F44E|
| Comparar con el otro usuario fuera de banda |
+-----+

```

MENSAJES

#0 [SERVER] ENVIADO [INTRODUCTION]

```

+--- VISTA CRIPTOGRAFICA (solo visible en este proceso) ---+
| Texto      : "Alicia"
| Nonce      : 53:32:43:01 | 00:00:00:00:00:00:00:00
| Plaintext  : 01 41 6C 69 63 69 61
| Cifrado    : EF 32 A2 55 C8 80 90
| Tag GCM    : 7A 5A F8 FE 60 94 80 CC E6 A1 72 81 3F DD E5 DC
+-----+

```

```

+--- VISTA DEL ATACANTE (bytes que viajan por la red) ---+
| Frame TCP (35 bytes):
| contador   : 00 00 00 00 00 00 00 00
| longitud   : 00 00 00 17
| payload    : EF 32 A2 55 C8 80 90 7A 5A F8 FE
| tag GCM    : 7A 5A F8 FE 60 94 80 CC E6 A1 72 81 3F DD E5 DC
+-----+

```

7. Despliegue con Docker

7.1 Dockerfile multi-etapa

```

FROM eclipse-temurin:11-jdk-alpine AS builder
WORKDIR /app
COPY src/ src/
RUN mkdir -p build/classes && \
    find src/main/java -name '*.java' | sort > build/main-s

```

```

sources.txt && \
    javac --release 11 -encoding UTF-8 -d build/classes @bu
ild/main-sources.txt && \
    jar --create --file build/encrypted-chat.jar \
        --main-class com.encryptedchat.ChatApplication \
        -C build/classes . && \
    jar --create --file build/observer.jar \
        --main-class com.encryptedchat.observer.SessionObse
rverApp \
        -C build/classes .

FROM eclipse-temurin:11-jre-alpine
WORKDIR /app
COPY --from=builder /app/build/encrypted-chat.jar .
COPY --from=builder /app/build/observer.jar .
ENTRYPOINT ["java", "-jar", "encrypted-chat.jar"]

```

La construcción multi-etapa usa una imagen de builder con el JDK completo para compilar y empaquetar, y luego copia solo los JARs resultantes a una imagen runtime mínima con solo el JRE. La imagen de builder (~300 MB) se descarta automáticamente; la imagen final ocupa aproximadamente 90 MB.

7.2 Escenario Docker Compose (simulación local)

```

services:
  server:
    build: .
    command: server --port 5050 --name Alicia
    ports:
      - "5050:5050"
    stdin_open: true
    tty: true

  client:
    build: .
    command: client --host server --port 5050 --name Bruno
    depends_on:
      - server

```

```
stdin_open: true
tty: true
```

Docker Compose crea una red virtual interna donde los contenedores se resuelven por nombre de servicio. El cliente usa `--host server` para conectar al contenedor del servidor.

7.3 Escenario dos maquinas reales

El observer siempre corre fuera de Docker en el host fisico. Los contenedores apuntan al observer via la variable `OBSERVER_HOST`:

```
# Maquina A: observer + servidor
Terminal 1: java -jar build/observer.jar
Terminal 2: docker run -it -p 5050:5050 \
            -e OBSERVER_HOST=host.docker.internal \
            andresb420/encrypted-chat:latest server --po
rt 5050 --name Alicia --observe

# Maquina B: cliente apuntando al observer de Maquina A
docker run -it \
    -e OBSERVER_HOST=IP_MAQUINA_A \
    andresb420/encrypted-chat:latest client --host IP_MAQUINA
_A --port 5050 --name Bruno --observe
```

`host.docker.internal` es la resolucion de Docker Desktop para el host fisico desde dentro de un contenedor (Windows y Mac). En Linux se usa la IP de la interfaz `docker0`.

8. Resultados y verificacion

8.1 Suite de pruebas automatizadas

La clase `CryptoSelfTest.java` incluye dos pruebas independientes:

Prueba 1 — Vector RFC 5869: valida que la implementacion de `Hkdf.extract()` y `Hkdf.expand()` produce exactamente los bytes especificados en el Test Case 1 del RFC. Esta es la unica prueba de correctitud matematica que no depende del entorno.

Prueba 2 — Conversacion end-to-end: levanta un hilo servidor y un hilo cliente en el mismo proceso JVM, conectados via loopback. Ejecuta el handshake completo, intercambia tres mensajes CHAT en ambas direcciones, verifica que el texto descifrado coincide con el enviado y ejecuta el cierre limpio del canal por parte del cliente.

```
./scripts/test.sh
# Salida esperada:
OK: todas las pruebas pasaron
```

8.2 Verificacion de propiedades de seguridad con el observer

Propiedad	Como se verifica	Evidencia en el observer
Confidencialidad	Comparar Vista Criptografica vs Vista del Atacante	El texto "Hola" aparece en VISTA CRIPTOGRAFICA; en VISTA DEL ATACANTE solo hay bytes sin estructura
Integridad	Modificar un byte del ciphertext en transito	<code>AEADBadTagException</code> al descifrar; conexion terminada con error
PFS	El secreto ECDH se borra tras el handshake	Observer muestra "[!] Este valor NUNCA sale por la red"; <code>Arrays.fill()</code> en el bloque finally
Ausencia de MITM	Verificar HMACs y fingerprint	Observer muestra "[OK] CLAVES COINCIDEN"; ambas terminales muestran el mismo fingerprint

8.3 Escenarios de prueba ejecutados

Escenario	Descripcion	Resultado
Test unitario HKDF	Vector oficial RFC 5869 Test Case 1	Correcto
Test loopback end-to-end	Servidor + cliente en el mismo proceso JVM	Correcto
Ejecucion local con observer	Tres terminales en la misma maquina	Funciona
Docker Compose	Dos contenedores en red virtual Docker	Funciona
Dos maquinas reales (LAN)	Maquinas fisicamente separadas	Funciona

Escenario	Descripcion	Resultado
Cierre desde el servidor	<code>/salir</code> tecleado por el servidor	Cierre limpio; cliente muestra "Alicia cerro la conversacion"
Cierre desde el cliente	<code>/salir</code> tecleado por el cliente	Cierre limpio; servidor muestra "Bruno cerro la conversacion"

```
#2 [SERVER] RECIBIDO [CLOSE]

+— VISTA CRIPTOGRAFICA (solo visible en este proceso) —+
| Nonce      : 43:32:53:01 | 00:00:00:00:00:00:02
| Plaintext: 03
| Cifrado   : C8
| Tag GCM   : D7 3C 2D D6 35 FD 09 F9 C8 C1 F1 C3 AA B9 0C C6
+—+

+— VISTA DEL ATACANTE (bytes que viajan por la red) —+
| Frame TCP (29 bytes):
| contador   : 00 00 00 00 00 00 00 02
| longitud   : 00 00 00 11
| payload    : C8 D7 3C 2D D6 35 FD 09 F9 C8 C1
| tag GCM    : D7 3C 2D D6 35 FD 09 F9 C8 C1 F1 C3 AA B9 0C C6
+—+

#2 [SERVER] ENVIADO [CLOSE]

+— VISTA CRIPTOGRAFICA (solo visible en este proceso) —+
| Nonce      : 53:32:43:01 | 00:00:00:00:00:00:02
| Plaintext: 03
| Cifrado   : 35
| Tag GCM   : 28 18 D0 5D 49 81 E4 C6 84 7E 02 68 70 13 6F 59
+—+

+— VISTA DEL ATACANTE (bytes que viajan por la red) —+
| Frame TCP (29 bytes):
| contador   : 00 00 00 00 00 00 00 02
| longitud   : 00 00 00 11
| payload    : 35 28 18 D0 5D 49 81 E4 C6 84 7E
| tag GCM    : 28 18 D0 5D 49 81 E4 C6 84 7E 02 68 70 13 6F 59
+—+
```

Paquete de 33 bytes con el mensaje "hola" en wireshark:

```
Transmission Control Protocol, Src Port: 5050, Dst Port: 57412, Seq: 173, Ack: 205, Len: 33
Data (33 bytes)
  Data: 000000000000000010000001507e40f779fdaFd2165b363930522dc21f934eb15c2
  [Length: 33]
```

Luego lo que percibe nuestro observer:

```

Agents
+----- VISTA CRIPTOGRAFICA (solo visible en este proceso) -----+
| Texto      : "Hola"
| Nonce      : 53:32:43:01 | 00:00:00:00:00:00:00:01
| Plaintext  : 02 48 6F 6C 61
| Cifrado    : 07 E4 0F 77 9F
| Tag GCM    : DA FD 21 65 B3 63 93 05 22 DC 21 F9 34 EB 15 C2
+-----+

General Concepts
+----- VISTA DEL ATACANTE (bytes que viajan por la red) -----+
| Frame TCP (33 bytes):
| contador   : 00 00 00 00 00 00 00 01
| longitud   : 00 00 00 15
| payload    : 07 E4 0F 77 9F DA FD 21 65 B3 63
| tag GCM    : DA FD 21 65 B3 63 93 05 22 DC 21 F9 34 EB 15 C2
+-----+

Talleres Docker/Ops

```

Coinciden los valores de contador, longitud, payload y tag GCM en el campo de data en la captura de wireshark. (Si se quiere comprobar, filtrar por puerto tcp.port == 5050 en la captura de wireshark)

Asi se veria una captura del trafico sin cifrar que llega al observer, en donde claramente se ve que hay acceso libre a todas las claves y mensajes sin ningun tipo de cifrado (Si se quiere comprobar, filtrar por puerto udp.port == 7777 en la captura de wireshark):

```

Frame 38: Packet, 372 bytes on wire (2976 bits), 0000 02 00 00 00 45 00 01 70 d1 af 00 00 80 11 69 cb .....E..p.....i.
Null/Loopback 0010 7f 00 00 01 7f 00 00 01 c5 83 1e 61 01 5c a6 c0 .....a...
Internet Protocol Version 4, Src: 127.0.0.1, Dst 0020 7b 22 72 6f 6c 65 22 3a 22 53 45 52 56 45 52 22 {"role": "SERVER"
User Datagram Protocol, Src Port: 50563, Dst Por 0030 2c 22 74 79 70 65 22 3a 22 4d 53 47 5f 53 45 4e , "type": "MSG.SEN
Data (340 bytes) 0040 54 22 2c 22 63 6f 75 6e 74 65 72 22 3a 22 31 22 T", "count er": "1"
Data [...] 0050 2c 22 6d 73 67 54 79 70 65 22 3a 22 43 48 41 54 , "msgTyp e": "CHAT
[Length: 340] 0060 22 2c 22 74 65 78 74 22 3a 22 68 6f 6c 22 2c 22 ", "text" : "hol", "
0070 6e 6f 6e 63 65 22 3a 22 35 33 3a 33 32 3a 34 33 nonce": " 53:32:43
0080 3a 30 31 3a 30 30 3a 30 30 3a 30 30 30 3a 30 30 3a 01:00:0 0:00:00:
0090 30 30 3a 30 30 3a 30 30 3a 30 31 22 2c 22 70 6c 00:00:00 :01:00:00:
00a0 61 69 6e 74 65 78 74 22 3a 22 30 32 20 36 38 20 aintext" : "02 68
00b0 36 46 20 36 43 22 2c 22 63 69 70 68 65 72 74 65 6F 6C", " cipherte
00c0 78 74 22 3a 22 36 30 20 32 34 20 45 33 20 41 42 xt": "60 24 E3 AB
00d0 22 2c 22 74 61 67 22 3a 22 35 42 20 45 30 20 46 ", "tag": " 5B E0 F
00e0 33 20 45 38 20 34 42 20 39 45 20 41 30 20 35 36 3 E8 40 9E A0 56
00f0 20 42 34 20 34 44 20 35 31 20 44 37 20 31 36 20 B4 4D 5 1 D7 16
0100 35 37 20 46 43 20 39 41 22 2c 22 66 72 61 6d 65 57 FC 9A ", "frame
0110 22 3a 22 30 30 20 30 30 20 30 30 20 30 20 30 20 30 30 20 30 20 30 20 30
0120 30 20 30 30 20 30 30 20 30 31 20 30 30 20 30 30 0 00 00 01 00 00

```

9. Conclusiones

9.1 Logros tecnicos

- **Implementacion completa sin dependencias externas:** toda la criptografia se construyo sobre `javax.crypto` y `java.security` del JDK 11, haciendo el

código completamente auditable línea por línea.

- **Perfect Forward Secrecy real:** las claves efímeras generadas en cada sesión y el borrado explícito de material clave garantizan que sesiones pasadas son irre recuperables.
- **Protocolo con confirmación activa de claves:** el sistema detecta activamente intentos de MITM mediante HMAC antes de que se intercambie ningún mensaje de chat.
- **Herramienta pedagógica del observador de sesión:** hace visible en tiempo real la brecha entre el mundo del participante legítimo (todos los valores criptográficos) y el mundo del atacante de red (ruido cifrado indescifrable), demostrando de forma concreta la efectividad del cifrado extremo a extremo.
- **Despliegue reproducible:** la imagen Docker publicada en Docker Hub permite demostrar el protocolo en cualquier máquina sin instalar Java ni compilar el código.

9.2 Limitaciones y trabajo futuro

Autenticación de identidad a largo plazo: ECDH con claves efímeras resiste MITM solo cuando los usuarios verifican el fingerprint manualmente por un canal alternativo. Un sistema productivo necesitaría certificados digitales o una PKI que vinculen la clave de sesión a una identidad verificada de forma automática.

Sesión única por conexión TCP: el protocolo no soporta reconexión sin nuevo handshake, multiplexación de conversaciones ni reanudación de sesión interrumpida.

Dependencia del usuario para el fingerprint: la seguridad final depende de que ambos usuarios comparen activamente el fingerprint. Un usuario que omita este paso es vulnerable a MITM activo.

Mejora criptográfica posible: la curva secp256r1 (NIST P-256) tiene una historia de desconfianza en parte de la comunidad criptográfica por sus parámetros de diseño. La curva X25519 (Curve25519, de Bernstein) ofrece seguridad equivalente con mayor transparencia en su diseño y menor riesgo de implementaciones con side-channels de tiempo.

9.3 Reflexión final

Implementar un protocolo de seguridad de red desde cero, sobre primitivas del JDK en lugar de sobre protocolos ya construidos como TLS, obliga a tomar y justificar cada decision de diseño: por que HKDF en lugar de usar directamente el secreto ECDH, por que claves efimeras, por que el transcript hash como sal, como construir nonces que no colisionen entre dos direcciones de comunicacion, como borrar material sensible de la memoria.

El observador de sesion transforma esas decisiones abstractas en algo concreto y verificable en una terminal. La Vista del Atacante muestra en cada mensaje que, independientemente del contenido del chat, los bytes que viajan por la red son estadisticamente uniformes e indescifrables sin la clave de sesion derivada en ese handshake especifico. Esta brecha visual entre lo que sabe el participante legitimo y lo que puede observar el atacante ilustra de forma inmediata, sin matematicas, el valor practico del cifrado extremo a extremo.

10. Referencias

1. **RFC 5869** — Krawczyk, H., Eronen, P. (2010). *HMAC-based Extract-and-Expand Key Derivation Function (HKDF)*. Internet Engineering Task Force.
2. **NIST SP 800-186** — Chen, L., et al. (2023). *Recommendations for Discrete Logarithm-based Cryptography: Elliptic Curve Domain Parameters*. National Institute of Standards and Technology.
3. **NIST SP 800-38D** — Dworkin, M. (2007). *Recommendation for Block Cipher Modes of Operation: Galois/Counter Mode (GCM) and GMAC*. National Institute of Standards and Technology.
4. **Diffie, W., Hellman, M.** (1976). *New Directions in Cryptography*. IEEE Transactions on Information Theory, 22(6), 644–654.
5. **Miller, V.** (1985). *Use of Elliptic Curves in Cryptography*. Advances in Cryptology — CRYPTO 85, Lecture Notes in Computer Science, vol 218. Springer, Berlin, Heidelberg.
6. **Ferguson, N., Schneier, B., Kohno, T.** (2010). *Cryptography Engineering: Design Principles and Practical Applications*. Wiley.
7. **McGrew, D., Viega, J.** (2004). *The Galois/Counter Mode of Operation (GCM)*. NIST submission.

8. **Oracle** (2021). *Java SE 11 Security Developer Guide*. Disponible en la documentacion oficial del JDK 11: `javax.crypto`, `java.security.spec`, `javax.crypto.Mac`.

```
Transmission Control Protocol, Src Port: 5050, Dst Port: 57412, Seq: 173, Ack: 205, Len: 33
Data (33 bytes)
  Data: 0000000000000000100000001507e40f779fdafd2165b363930522dc21f934eb15c2
  [Length: 33]
```